Admin Views

Controller Integration Guide

This document describes the public functionality exposed by every Java Swing view in `com.admin.views`.

Covered classes:

- `AdminView`
- `CreateRestaurantView`
- `ShowRestaurantsView`
- `ShowCustomersView`

1. General controller rules

1. Create views and interact with Swing components on the Swing Event Dispatch Thread (EDT).
2. The controller attaches listeners exposed by the view.
3. The controller calls services and repositories; views do not access the database.
4. The controller supplies lists and models through the view's public methods.
5. Always check selected-model getters for `null`.

```
Console/application setup
    creates repositories and services
            |
            v
Controller attaches view listeners
            |
            v
View reports user action
            |
            v
Controller calls service and updates view
```

Selection warning: Calling `setRestaurants(...)` or `setCustomers(...)` rebuilds the table data and clears the current selection. Read the selected model before refreshing.

Typical window setup

```
SwingUtilities.invokeLater(() -> {
    AdminView view = new AdminView();
    // Attach listeners before displaying the window.
    view.setVisible(true);
});
```

2. AdminView

`AdminView` is the main admin dashboard. It exposes buttons for opening the other admin screens. The view does not create those screens itself.

Method	Purpose
`addCreateRestaurantListener(ActionListener)`	Called when Create Restaurant is pressed.
`addShowRestaurantsListener(ActionListener)`	Called when Show Restaurants is pressed.
`addShowCustomersListener(ActionListener)`	Called when Show Customers is pressed.

Controller/application example

```
AdminView adminView = new AdminView();

adminView.addCreateRestaurantListener(event -> {
    createRestaurantView.setVisible(true);
    adminView.setVisible(false);
});

adminView.addShowRestaurantsListener(event -> {
    showRestaurantsController.refreshRestaurantList();
    showRestaurantsView.setVisible(true);
    adminView.setVisible(false);
});

adminView.addShowCustomersListener(event -> {
    showCustomersView.setCustomers(customerService.getAllCustomers());
    showCustomersView.setVisible(true);
    adminView.setVisible(false);
});

adminView.setVisible(true);
```

3. CreateRestaurantView

This view collects owner-account and restaurant information. It also contains a modal picker for selecting an existing user.

Input getters

Owner input	Restaurant input
`getUsernameInput()`	`getRestaurantNameInput()`
`getEmailInput()`	`getAddressInput()`
`getPasswordInput()`	`getCityInput()`
`getPhoneInput()`	`getPostalCodeInput()`
	`getRestaurantPhoneInput()`
	`getRestaurantEmailInput()`
	`getCuisineTypeInput()`

Actions and messages

Method	Purpose
`addCreateListener(ActionListener)`	Registers the controller's Create action.
`showMessage(String)`	Displays controller feedback using a message dialog.
`setOwnerFields(UserModel)`	Copies a user's username, email, password, and phone into the owner form.

Existing-user picker

Method	Purpose
`addShowUsersListener(ActionListener)`	Registers work that runs before the user-picker dialog opens. Use it to refresh the list.
`clearUsers()`	Removes all current picker entries.
`addUser(UserModel)`	Adds one user to the picker. The picker displays the username.
`getSelectedUser()`	Returns the selected user, or `null`.

> Show-user listeners execute in registration order and finish before the modal picker is displayed.

Controller example

```
view.addShowUsersListener(event -> {
    view.clearUsers();
    for (UserModel user : userService.getAllUsers()) {
        view.addUser(user);
    }
});

view.addCreateListener(event -> {
    UserModel user = new UserModel();
    user.setUsername(view.getUsernameInput());
    user.setUserEmail(view.getEmailInput());
    user.setUserPassword(view.getPasswordInput());
    user.setUserPhone(view.getPhoneInput());

    StoreManagerModel restaurant = new StoreManagerModel();
    restaurant.setName(view.getRestaurantNameInput());
    restaurant.setAddress_line1(view.getAddressInput());
    restaurant.setCity(view.getCityInput());
    restaurant.setPostal_code(view.getPostalCodeInput());

    // Call the service and translate its result into view.showMessage(...).
});
```

4. ShowRestaurantsView

This view displays restaurants in an editable table. Each row ends with a three-dot menu containing Save Changes, Show More, and Remove.

Data methods

Method	Purpose
`setRestaurants(List<StoreManagerModel>)`	Replaces the table contents. Passing `null` produces an empty table.
`getRestaurants()`	Returns a new list containing the current model objects.
`getSelectedRestaurant()`	Commits any active cell edit and returns the selected row's model, or `null`.

Editable cells update the corresponding `StoreManagerModel` immediately. The list itself is copied, but the contained model objects are shared.

Row action listeners

Method	Triggered by
`addSaveChangesListener(ActionListener)`	Save Changes in the selected row's menu.
`addShowMoreListener(ActionListener)`	Show More in the selected row's menu.
`addRemoveListener(ActionListener)`	Remove in the selected row's menu.
`showMessage(String)`	Controller feedback.

Read-only details dialog

`showRestaurantDetails(Component, StoreManagerModel)` displays every restaurant field in a scrollable application-modal dialog. The parent window is blocked until the dialog closes. The overload `showRestaurantDetails(StoreManagerModel)` creates the same dialog without a parent component.

Controller example

```
view.setRestaurants(service.getAllRestaurants());

view.addSaveChangesListener(event -> {
    StoreManagerModel selected = view.getSelectedRestaurant();
    if (selected == null) return;
    service.updateRestaurant(selected);
    view.setRestaurants(service.getAllRestaurants());
});

view.addShowMoreListener(event -> {
    // Read selection BEFORE any refresh, because refreshing clears selection.
    StoreManagerModel selected = view.getSelectedRestaurant();
    ShowRestaurantsView.showRestaurantDetails(view, selected);
});

view.addRemoveListener(event -> {
    StoreManagerModel selected = view.getSelectedRestaurant();
    if (selected == null) return;
    service.deleteRestaurant(selected);
    view.setRestaurants(service.getAllRestaurants());
});
```

5. ShowCustomersView

This view displays a read-only customer table containing full name, city, state, postal code, and address. It reports a double-click but intentionally does not decide what response should occur.

Method	Purpose
`setCustomers(List<CustomerModel>)`	Replaces all table rows.
`getCustomers()`	Returns a new list containing the displayed customer objects.
`getSelectedCustomer()`	Returns the selected customer's complete model, or `null`.
`addCustomerDoubleClickListener(ActionListener)`	Registers controller logic invoked after a customer row is double-clicked.

Controller example

```
view.setCustomers(customerService.getAllCustomers());

view.addCustomerDoubleClickListener(event -> {
    CustomerModel selected = view.getSelectedCustomer();
    if (selected == null) return;

    // The controller decides how complete information is displayed.
    // For example: open a modal customer-details dialog.
});
```

> The view does not automatically open a details dialog. This is intentional: it only supplies the selected model and event hook.

6. Window navigation and closing

All four views use `JFrame.DISPOSE_ON_CLOSE`. A controller or application coordinator can restore the dashboard when a child view closes.

```
childView.addWindowListener(new WindowAdapter() {
    @Override
    public void windowClosed(WindowEvent event) {
        adminView.setVisible(true);
    }
});
```

7. Quick reference

View	Controller supplies	View reports
`AdminView`	Navigation listeners	Dashboard button clicks
`CreateRestaurantView`	User list, owner model, listeners	Form input and Create click
`ShowRestaurantsView`	Restaurant list and action listeners	Edited model, selected row, row-menu actions
`ShowCustomersView`	Customer list and double-click listener	Selected complete customer model